

Native Godot Battle Engine Port Plan

A focused path from the existing Pokémon Showdown REST behavior to a local, callable GDScript API—without porting the frontend, networking, or multiplayer session layer.

Decision: build a synchronous BattleEngine that extends RefCounted, owns one isolated battle session, accepts typed Dictionaries, and returns deep-copied battle responses. The current REST server is the development oracle only; the shipped game runs no JavaScript, Node server, HTTP call, token, WASM, or native extension.

What “done” means

First playable target

- Generation 9, one-on-one singles, teams of 1–6.
- Start, move, voluntary switch, forced switch, forfeit, win/tie.
- Stable member IDs, HP/max HP, status, move PP, active/fainted flags.
- Seeded battle RNG and separate deterministic AI RNG.
- A deliberately small, declared move/type/status subset first.

Explicit non-goals

- No browser UI, REST routes, CORS, encryption, replay tokens, or remote sessions.
- No Team Preview, doubles, competitive legality checker, or every generation in the MVP.
- No silent approximation: an unported mechanic returns unsupported_mechanic.
- No battle math inside BattleScene.gd or the verification GUI.

The API the game calls

```
var engine := BattleEngine.new()
var opened := engine.start_battle(start_input, {
  "battle_seed": 1234, "ai_seed": 5678
})
var advanced := engine.apply_action({"type": "move", "moveIndex": 1})

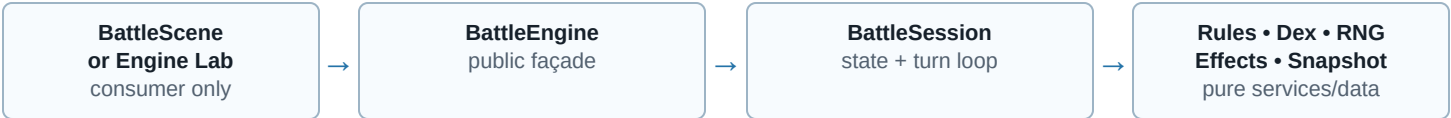
# Both calls return:
{"ok": true, "response": {
  "revision": 1, "phase": "awaiting_player", "events": [...],
  "request": {...}, "parties": {"player": [...], "opponent": [...]},
  "result": null
}}
```

Keep the current REST input/action field names so fixtures and adapters stay simple. Use GDScript-style method names. The engine advances the chosen player action plus all required AI decisions, then stops at the next player decision or battle end.

Port the contract, replace the transport

Keep from battle_server	Replace for native Godot	Do not port
Team/action validation, canonical defaults, stable IDs	One BattleSession held in memory	Next.js routes and Request/Response handling
Decision requests, party snapshots, result semantics	export_snapshot() / import_snapshot()	Encrypted stateToken, CORS, body limits
Deterministic turn flow and separate AI randomness	Godot Resources/JSON plus pure GDScript rules	Showdown protocol privacy and multiplayer concerns

Dependency direction



Architecture gate: two engine instances can run simultaneously with no shared state, the engine can run headless with no scene tree, and every returned Dictionary is safe for the caller to mutate.

Milestones 0–2: Contract, Data, Session

Each milestone ends with both a headless test gate and a visible button-driven check. Do not start the next milestone while either gate is red.

0 Freeze semantic parity fixtures + build the lab shell

GATE: ORACLE LOCKED

IMPLEMENT

- Add tests/fixtures/battle_engine/ with fixed inputs, actions, seeds, and expected semantic outputs.
- Generate them once from the server's lower-level Showdown adapter/test seam. Exclude token, HTTP status, API version, and random battle ID.
- Document a normalizer for event text and field-name differences; never hand-edit expected battle math.
- Create a test runner and empty BattleEngineLab scene that can load a preset.

AUTOMATED PROOF

- Fixture JSON parses and its action sequence is complete.
- Every expected player decision boundary has a request.
- Golden cases cover reduced HP, switch, knockout, simultaneous replacement, forfeit, and natural completion.
- Baseline remains green: 89 server tests and the two current Godot battle smoke tests.

MANUAL GUI PROOF

- Open the lab, choose **Fixture Viewer**, click **Load**.
- Input, expected actions, and expected final winner appear.
- The checker says **FIXTURE VALID**; engine controls remain disabled at this stage.

1 Canonical data, validation, stats, and deterministic RNG

GATE: CORRECT NUMBERS

IMPLEMENT

- BattleDex loads a small audited species/move/type dataset from Godot Resources or JSON.
- BattleValidator enforces 1–6 members, 1–4 moves, level 1–100, IDs, IV/EV bounds, and supported mechanics.
- Fill current defaults: first ability, no item, Serious nature, IV 31, EV 0.
- BattleStats implements Gen 9 stat formulas, Shedinja HP = 1, and starting-health resolution.
- BattleRng has serializable state; battle and AI streams are separate.

AUTOMATED PROOF

- Valid/invalid DTO table tests mirror schemas and team tests.
- Known base stats produce exact HP/Attack/etc. at levels 1, 50, and 100.
- Normalized health 0 stays on the roster but cannot become active; positive health rounds and clamps to 1...max HP.
- Same seed gives the same sequence; exporting/restoring RNG continues at the same next value.
- Unknown data fails loudly—never defaults to neutral behavior.

MANUAL GUI PROOF

- Select **Reduced HP + Fainted Bench** and click **Start**.
- Roster cards show exact level, HP/max HP, percentage, and fainted flag.
- Click **Bad Input**: see a stable error code/message, not a crash.
- Reset with the same seeds: displayed values and RNG preview match.

2 Session state, response builder, and legal decision requests

GATE: CALLABLE API

IMPLEMENT

- BattleEngine owns one BattleSession: two sides, field, turn, revision, phase, and result.
- Start with the first living roster member. Preserve caller member IDs and original roster order.
- BattleSnapshotBuilder returns request, public party stats, PP, switches, events, and result.
- Validate typed move, switch, and forfeit actions. Implement forfeit fully; moves may be a no-damage stub only in this gate.

AUTOMATED PROOF

- Start returns revision 0, phase awaiting_player, correct active ID, move PP, and legal switch options.
- Unavailable, mixed-shape, or stale-phase actions return stable errors.
- Forfeit returns phase ended, opponent winner, and no next request.
- Mutating a returned response cannot alter engine state.
- Two sessions interleave actions without state leakage.

MANUAL GUI PROOF

- Click **Start**: move and switch buttons are built only from response.request.
- Click an illegal injected action: error panel turns red but state does not change.
- Click **Forfeit**: controls disable and result reads **opponent / forfeit**.

Scope rule: the first dataset should contain only the species, moves, types, natures, statuses, abilities, and items needed by the fixtures. Expand coverage through tests; do not bulk-port a catalog before the execution model is proven.

Milestones 3–6: Turns, Effects, Integration

The goal is a vertical slice that is provably correct, then a repeatable way to add mechanics without turning the engine into one giant match statement.

3 Turn queue, damage pipeline, PP, and automatic AI

GATE: ONE REAL TURN

IMPLEMENT

- Pipeline: validate → queue → priority → speed → seeded tie-break → execute → faint check → next request.
- Port accuracy, damage roll, critical hit, STAB, type effectiveness/immunity, stat stages, and PP consumption in explicit functions.
- After a player choice, run all required opponent decisions with a separate deterministic AI RNG until the next player boundary.
- Initial vertical moves: normal damage, priority damage, type damage/immunity, status, and stat-lowering examples.

AUTOMATED PROOF

- Priority beats speed; speed beats lower speed; tied order is seed-repeatable.
- Golden seeds verify exact damage, accuracy, crit, STAB, effectiveness, immunity, PP, and no negative HP.
- AI reaches the same decision and end state after reset/replay.
- One action increments revision once even though the AI also acts.

MANUAL GUI PROOF

- Select **Priority vs Speed**; click the slower side's priority move.
- Event log shows the player acting first, both HP changes, and PP decrement.
- Click **Replay Same Seeds**: state hash and every event match.
- Select **Type Immunity**: HP remains unchanged and the reason is visible.

4 Switching, fainting, replacements, and battle result

GATE: COMPLETE BATTLE

IMPLEMENT

- Voluntary switch, forced player replacement, deterministic AI replacement, and simultaneous replacement boundary.
- Retain fainted roster members, exclude them from switch options, and keep stable IDs through internal reordering.
- Resolve player win, opponent win, tie, forfeit, simultaneous self-knockout, and the 500-turn safety stop.
- Emit structured events with display text; the UI never infers lifecycle from HP alone.

AUTOMATED PROOF

- Mirror current server regressions: voluntary reorder, forced replacement after AI KO, simultaneous replacements, forfeit, natural finish, and self-KO winner.
- A forced-switch request rejects moves and offers only living bench IDs.
- Ended responses have a result and no request; further actions fail without mutation.
- Original roster order remains stable in every snapshot.

MANUAL GUI PROOF

- Run **Voluntary Switch**: click a bench ID; active badge moves and HP stays attached to the correct ID.
- Run **Forced Switch**: after KO, only switch controls remain enabled.
- Choose the final replacement and play to completion; winner/reason appear and all controls lock.

5 Effect hooks, statuses, abilities, items, and field rules

GATE: SAFE TO EXPAND

IMPLEMENT

- Add ordered hooks such as `before_action`, `modify_power`, `on_hit`, `after_damage`, `residual`, and `switch_in`.
- Effects are small GDScript strategy objects/resources; session code owns ordering, not individual moves.
- Add one family at a time: major status, volatile status, abilities, items, weather/terrain, then unusual move rules.
- Maintain a coverage manifest with **supported**, **blocked**, or **not started**—never “probably works.”

AUTOMATED PROOF

- Every new effect gets a focused unit test plus at least one Showdown parity fixture.
- Hook order is asserted explicitly, including switch-in and end-turn residuals.
- Unsupported effects fail during validation before battle start.
- A catalog test ensures every exposed move/ability/item maps to an implemented effect.

MANUAL GUI PROOF

- Use **Status + Residual**; badges, turn log, HP ticks, and disabled moves update after each click.
- Inspector shows the ordered hook trace for the selected turn.
- Use **Unsupported Data**; Start is rejected with the exact missing mechanic ID.

6 Save/restore and connect the real BattleScene

GATE: GAME READY

IMPLEMENT

- Versioned, data-only snapshot includes session, battle RNG, AI RNG, revision, and coverage version.
- Add `BattleController.gd`: adapts `CollectionSystem` launch data, owns the engine, and applies final HP/status back only when battle ends.
- Bind `Fight` → request moves, `Party` → request switches, `Run` → forfeit. Keep `Bag` disabled until item-in-battle rules exist.
- `BattleScene` continues to own presentation, timing, camera, and HUD only.

AUTOMATED PROOF

- Save after any decision, restore, take the same action, and compare exact response/state hash.
- Old/invalid snapshot versions return stable errors.
- Existing battle start and UI layout smoke tests stay green.
- Integration test proves `CollectionSystem` is untouched mid-battle and updated once at result.

MANUAL GUI PROOF

- In the lab: play, **Save Snapshot**, act, restore, repeat action; result matches.
- In `BattleScene`: select moves/switches through the real HUD and inspect returned stats.
- Finish battle, return to overworld, and confirm roster HP/status match the final response.

Manual Battle Engine Lab

A small developer scene makes every gate observable without relying on the finished game UI. It must call the same public engine methods as production and contain zero battle rules.

Recommended files

```
battle/engine/
BattleEngine.gd      # public RefCounted façade
BattleSession.gd     # mutable battle state
BattleValidator.gd   # input/action checks
BattleStats.gd       # stat + damage calculations
BattleRng.gd         # serializable PRNG
BattleTurnQueue.gd   # ordering and execution
BattleEffects.gd     # ordered hook runner
BattleSnapshotBuilder.gd # caller-safe response
data/                # Resources / JSON + manifest

battle/dev/
BattleEngineLab.tscn
BattleEngineLab.gd

tests/battle_engine/
battle_engine_suite.tscn
*_test.gd
tests/fixtures/battle_engine/*.json
```

Scene layout

```
BattleEngineLab (Control)
├── Margin / VBox
│   ├── Header (HBox)
│   │   ├── Preset (OptionButton)
│   │   ├── BattleSeed / AiSeed
│   │   └── Start / Reset / Replay
│   └── Main (HSplitContainer)
│       ├── Parties (player + opponent tables)
│       ├── Actions
│       │   ├── MoveGrid (buttons 1-4)
│       │   ├── SwitchList (stable member IDs)
│       │   └── Forfeit
│       ├── RequestAndState (read-only JSON)
│       ├── EventLog (RichTextLabel)
│       └── ExpectedChecks (green/red rows)
```

One update path: every Start/action returns a response, then `render_response(response)` rebuilds cards, enabled controls, request view, events, result, and expected checks. Never update visible HP optimistically.

GUI implementation instructions

- Create the scene tree above with containers so it works at the project's 1152x648 viewport. Give the party/event areas scroll containers.
- In `_ready()`, instantiate one `BattleEngine`, load preset JSON, connect buttons, and leave action controls disabled until a valid request arrives.
- Attach the complete action Dictionary as button metadata. A press calls only `engine.apply_action(button.get_meta("action"))`.
- For each response, show revision/phase; member ID, species, level, HP/max HP/percent, status, active/fainted; moves and PP; request; result; and event text.
- Each preset contains a short sequence of expected predicates (not screenshots): active ID, phase, legal action types, HP/PP deltas, event kinds, and winner. Re-evaluate after every click and show **PASS** or **FAIL**.
- Add **Copy Response**, **Save Snapshot**, **Load Snapshot**, and **Replay Same Seeds**. The lab may inspect data; it may not calculate damage, choose AI moves, or repair engine output.

The seven-click verification circuit

Preset / click	Expected result → proof
1. Bad Input → Start	Stable code; no state/revision → safe boundary
2. Reduced HP → Start	Exact HP; zero-HP bench retained → canonical stats
3. Priority → move 1	Actor first; exact HP/PP; AI; request → real turn loop
4. Same preset → Replay	Identical events and state hash → determinism
5. Voluntary Switch	Active badge moves; IDs/HP follow → switch semantics
6. Forced Switch	Moves disabled; living bench only → decision boundary
7. Finish / Forfeit	Winner/reason; no request; locked → result contract

Run on every milestone

```
godot --headless --path . --scene \
res://tests/battle_engine/battle_engine_suite.tscn

godot --headless --path . --scene \
res://tests/battle_start_smoke_test.tscn

godot --headless --path . --scene \
res://tests/battle_ui_layout_smoke_test.tscn
```

Launch the lab manually:

```
godot --path . --scene \
res://battle/dev/BattleEngineLab.tscn
```

MVP release gate

- All headless engine, parity, save/restore, and existing smoke tests pass.
- All seven manual circuits are green.
- No engine file imports UI, networking, Node.js, or CollectionSystem.
- Two sessions are independent; returned data is deep-copied.
- Supported-mechanics manifest matches exposed data.
- BattleScene consumes requests and final stats through BattleController only.

Expansion rule: one mechanic family per change, with a focused unit test, a parity fixture, and a lab preset when humans need to observe it.